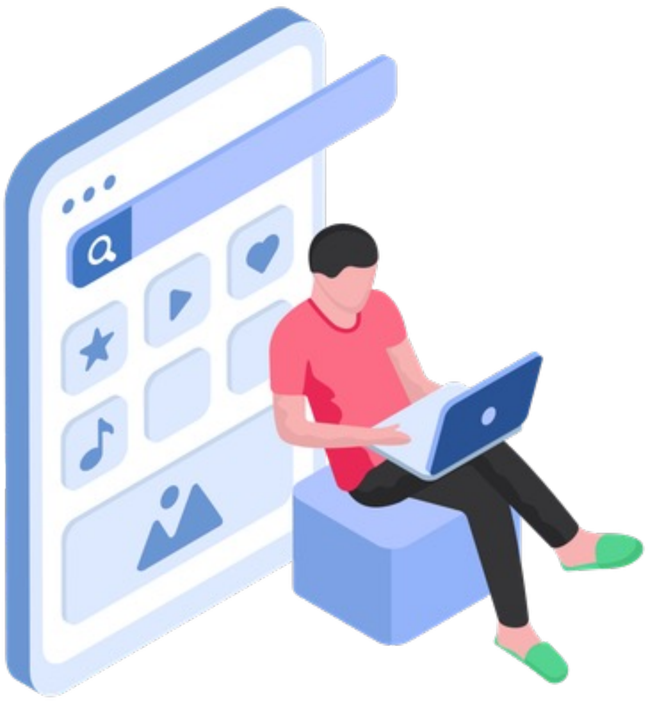
Week - 3

Android Fundamentals & UI Basics



Mudassir Ahmad Khan
Lecturer CS/IT
mudassirahmad@aup.edu.pk

Android Fundamentals & UI Basics

Android Component

Android app = made of 4 main parts

- **Activities** → screens user sees
- **Intents** → messages between screens
- **Views** → buttons, text, images on screen
- **Resources** → images, strings, colors (kept separately)

XML Basics in Android Studio

What is XML in Android?

XML (eXtensible Markup Language) is used to **design the app screen (UI)**.

In Android:

- **XML = Design / Layout**
- **Java / Kotlin = Logic / Function**

So:

- Buttons, Text, Images → **XML**
- Click actions, calculations → **Java/Kotlin**

Where XML is Used in Android Studio

You will mostly find XML files in:

-> res → layout

Examples:

- `activity_main.xml`
- `login_screen.xml`
- `item_row.xml`

Structure of an XML File

Basic structure:

```xml
<?xml version="1.0" encoding="utf-8"?>
<LayoutName

    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <!-- UI Components here -->

</LayoutName>
```

Important points:

- <?xml ...?> → XML declaration
- <LayoutName> → Parent layout
- Everything is written **inside tags**

Layouts in Android

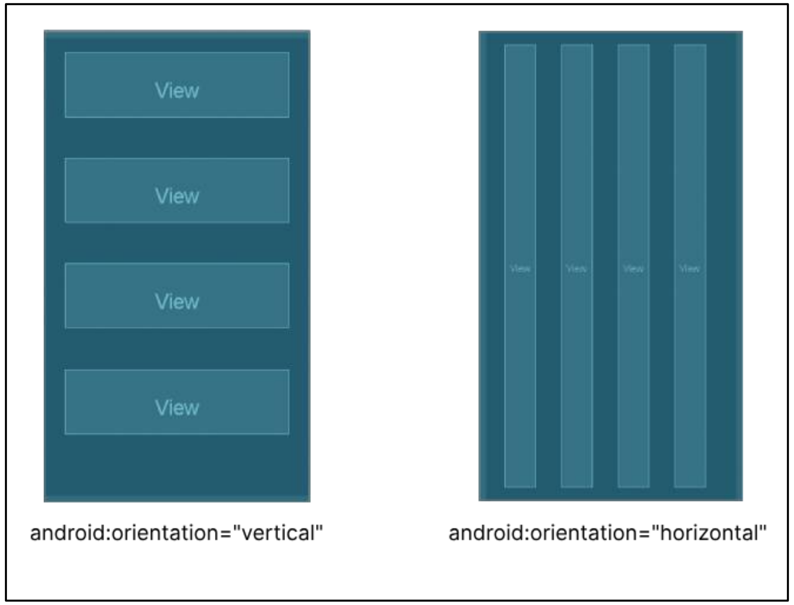
Layouts decide **how views are arranged** on the screen.

1. LinearLayout

Arranges items **in a row or column**

```xml
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">
```

- vertical → top to bottom
- horizontal → left to right



Layouts in Android

Layouts decide **how views are arranged** on the screen.

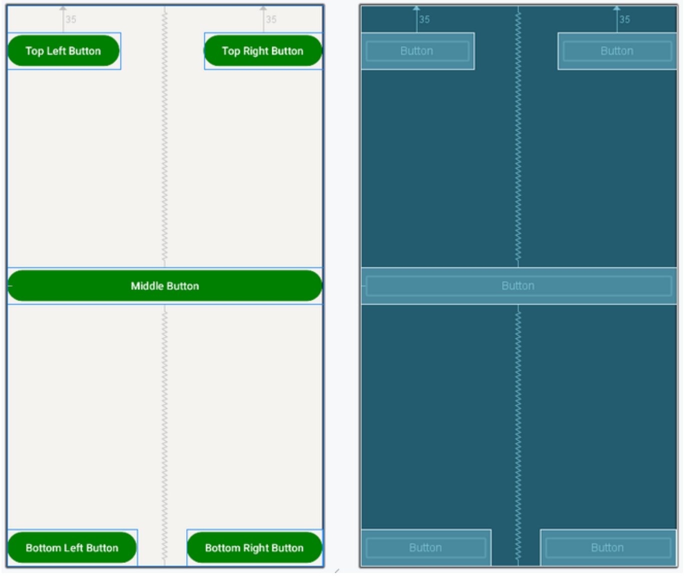
2. RelativeLayout

Views are placed **relative to each other**

```
<RelativeLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">
```

Example:

- below another view
- center of screen



Layouts in Android.

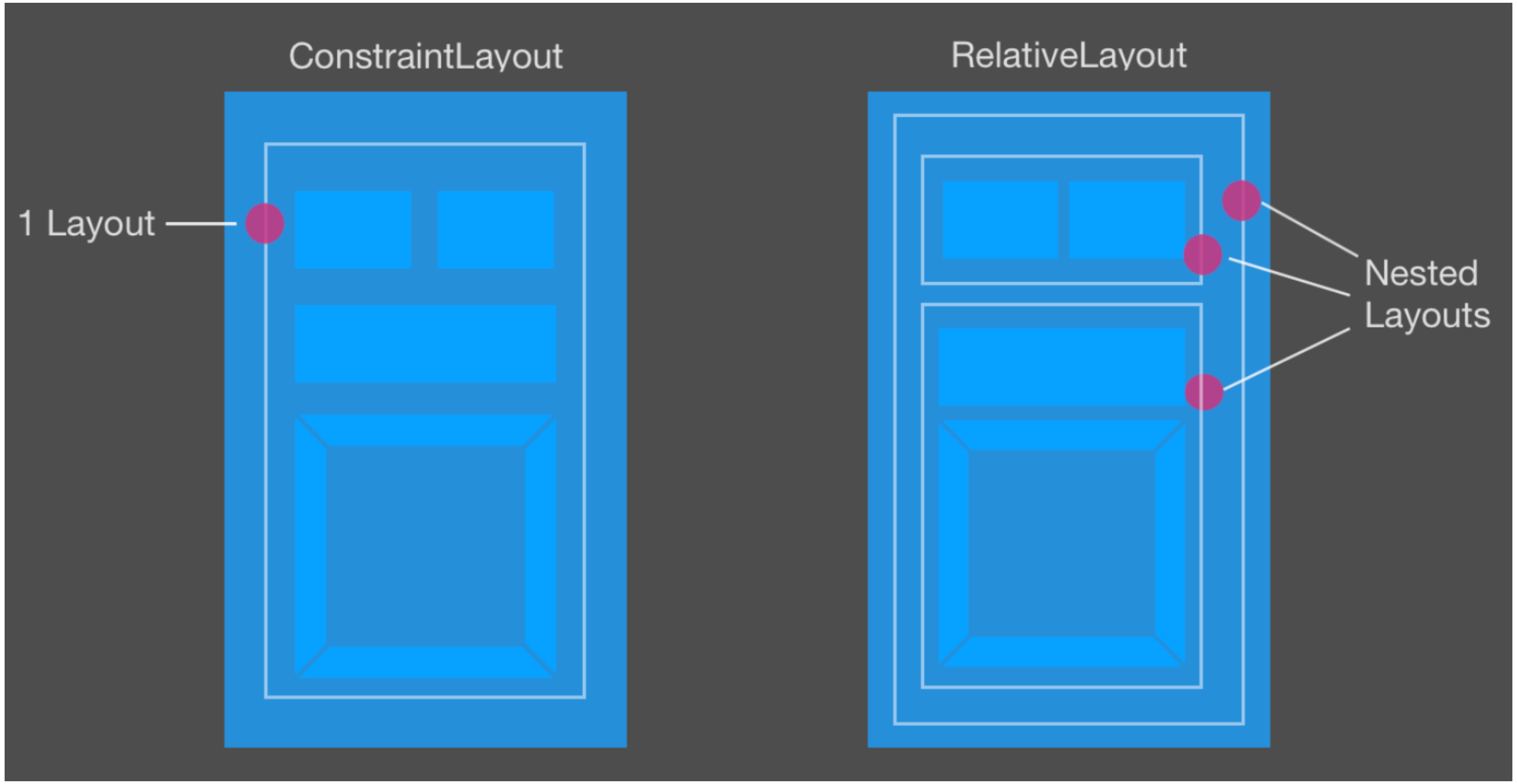
3. ConstraintLayout (Most Used)

- Flexible and powerful
- Used in modern apps

```
<androidx.constraintlayout.widget.ConstraintLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">
```

- Best for complex UI
- Recommended by Google

ConstraintLayout and RelativeLayout



Common UI Elements (Views)

1. TextView (Text)

Used to show text.

```xml
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello Android"/>
```

Common UI Elements (Views)

2. **EditText (Input field)**
   Used for user input.

```xml
<EditText
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter your name"/>
```

Common UI Elements (Views)

3. **Button**

   Used for actions.

```xml
<Button
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Login"/>
```

Common UI Elements (Views)

4. **ImageView**

   Used to show images.

```xml
<ImageView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:src="@drawable/logo"/>
```

Layout Width & Height

Values:

- match_parent → take full space
- wrap_content → take only required space
- dp → fixed size (recommended)

Example

```
android:layout_width="200dp"
android:layout_height="50dp"
```

Padding vs Margin

- **Padding**

  Space **inside** the view

```
android:padding="10dp"
```

- **Margin**

  Space **outside** the view

```
android:layout_margin="10dp"
```

IDs in XML

IDs connect XML with Java/Kotlin code.

```
android:id="@+id/btnLogin"
```

Later in Java/Kotlin:

```
Button btn = findViewById(R.id.btnLogin);
```

Colors in XML

Use colors from `colors.xml`

```
android:textColor="@color/black"
android:background="@color/purple_500"
```

Text Size & Style

```
android:textSize="18sp"
android:textStyle="bold"
```

dp (density-independent pixel)

Used for:

- Layout size
- Height / width
- Margin / padding
- Button size
- Image size

sp (scale-independent pixel)

Used for:

- **Text size only**